EasyDocs Backend Architecture Blueprint

Spatial Workspace Models & API Specifications

1. Database Schema (Django Models)

The spatial workspace relies on three core entities stored in PostgreSQL. These models define the infinite canvas, bounded visual groups, and positional assets.

Model	Field	Type	Meaning / Example
Board The infinite canvas	`id`	UUID	Primary key (secure routing)
	`company`	ForeignKey	Links board to the tenant/organization
	`board_type`	Char	'PRIMARY' (Internal) or 'CLIENT' (Deal Room)
Galaxy Visual frame/zone	`board`	ForeignKey	The board this frame belongs to
	`name`	Char	"Q4 Marketing Assets", "Legal Docs"
	`x_position, y_position`	Float	Coordinates on canvas (e.g., X: 500, Y: -200)
	`width, height`	Float	Dimensions of the bounding box
Asset Files & Links	`board, galaxy`	ForeignKey	Context (can float on board or sit in galaxy)
	`title`	Char	"Employee_Handbook.pdf"
	`file`	FileField	Local MEDIA_ROOT storage path
	`x_position, y_position`	Float	Exact card placement coordinates
	`uploaded_by`	ForeignKey	User tracking and RBAC checks

Signal Automation: A Django `post_save` signal automatically generates a default 'PRIMARY' Board whenever a new Company is registered.

2. REST API Endpoints

These endpoints drive the React frontend, allowing the infinite canvas to fetch spatial data and persist drag-and-drop actions.

GET `/api/workspace/board/primary/`

Purpose: Fetches the full canvas state on load. Returns the board object along with nested lists of all related Galaxies and Assets.

POST `/api/workspace/galaxy/`

Purpose: Creates a new bounding box on the canvas. Management draws a square, React calculates X/Y/Width/Height, and sends the payload to save it.

POST `/api/workspace/asset/`

Purpose: File upload endpoint. Accepts FormData (a physical file and target X/Y coordinates). Placed in the center of the user's current viewport.

PATCH `/api/workspace/asset/{id}/move/`

Purpose: Drag-and-drop persistence. Extremely lightweight endpoint that only accepts `x_position` and `y_position` to update location instantly.

Architecture Flow



3. API Responses & Frontend Usage

To render the entire canvas in one render cycle without cascading network requests, the primary GET request uses nested serialization.

Sample Response Payload


```
      { "id": "b1a2c3d4-...", "name": "Acme Corp Primary Board", "board_type":
"PRIMARY", "galaxies": [ { "id": "g1...", "name": "Design Assets", "x_position": 100.0,
"y_position": 250.0, "width": 600.0, "height": 400.0 } ], "assets": [ { "id": "a1...",
"title": "logo.png", "file": "/media/board_assets/logo.png", "x_position": 150.0,
"y_position": 300.0, "galaxy": "g1..." } ] }
```


Where This is Used (Frontend Integration)

Canvas Bootstrapping

The JSON payload feeds directly into the `react-zoom-pan-pinch` component. Galaxies map to absolute-positioned border divs; Assets map to shadcn/ui Cards.

Drag Persistence

When an `onDragEnd` event fires on an Asset, React calculates the new delta, updates local UI state instantly, and fires the PATCH endpoint silently in the background.